

Домашнее задание

Observability в Kubernetes

Выберите один из двух вариантов

Вариант	Инструмент	Что нужно развернуть	Оценка
Вариант 1	kubectl	Prometheus + Grafana через raw-манифесты	8 баллов
Вариант 2	Helm	kube-prometheus-stack + ServiceMonitor + алерты	10 баллов

Общие требования для обоих вариантов: использовать свой проект (можно *crud-example*); кластер поднять локально через *kind*; сдать репозиторий с манифестами/чартом и *README* со скриншотами.

Вариант 1 — kubectl

8 баллов

Цель

Добавить observability к своему проекту, развернув Prometheus и Grafana через сырые Kubernetes-манифесты (kubectl apply). Prometheus должен автоматически обнаруживать поды приложения и собирать метрики; Grafana — отображать их на дашборде.

Что нужно сделать

- Добавить в своё приложение эндпоинт /metrics в формате Prometheus.
- Создать namespace monitoring и задеплоить в него Prometheus (Deployment, Service, ConfigMap со scrape-конфигом, RBAC для доступа к Kubernetes API).
- Настроить kubernetes_sd_configs в scrape-конфиге так, чтобы Prometheus автоматически находил поды вашего приложения по label.
- Задеплоить Grafana (Deployment, Service) с автоматическим провижнингом Prometheus как datasource через ConfigMap.
- Создать в Grafana дашборд с минимум 3 панелями (RPS, латентность, ошибки). Экспортировать дашборд в JSON и сохранить в репозиторий.

Структура репозитория

Файл / папка	Назначение
app/	Исходный код приложения с добавленным /metrics
k8s/namespace.yaml	Namespace вашего приложения
k8s/app.yaml	Deployment + Service вашего приложения
k8s/monitoring/namespace.yaml	Namespace monitoring
k8s/monitoring/prometheus-config.yaml	ConfigMap: scrape-конфиг Prometheus
k8s/monitoring/prometheus.yaml	Deployment + Service + RBAC для Prometheus
k8s/monitoring/grafana.yaml	Deployment + Service + datasource ConfigMap для Grafana
k8s/monitoring/grafana-dashboard.json	Экспортированный JSON дашборда
README.md	Инструкция по запуску + скриншоты

Критерии оценки

Критерий	Баллы
Приложение отдаёт метрики на /metrics	2
Prometheus запущен, target приложения в состоянии UP (скриншот)	2
Grafana запущена, Prometheus подключён как datasource	2
Дашборд с ≥ 3 панелями, экспортирован в	2

JSON в репозиторий	
Итого	8

Скриншоты в README: (1) Prometheus → Status → Targets — ваш job в состоянии UP; (2) Grafana — дашборд с реальными данными.

Вариант 2 — Helm

10 баллов

Цель

Развернуть production-ready стек observability через Helm-чарт kube-prometheus-stack. Стек включает Prometheus, Alertmanager, Grafana, node-exporter и kube-state-metrics — всё устанавливается одной командой. Автообнаружение метрик приложения настраивается через CRD ServiceMonitor, алерты — через CRD PrometheusRule, конфигурация вынесена в values.yaml.

Что нужно сделать

- Добавить в своё приложение эндпоинт /metrics в формате Prometheus.
- Установить kube-prometheus-stack из репозитория prometheus-community через helm install с кастомным файлом monitoring-values.yaml (настроить ресурсы, пароль Grafana, NodePort или port-forward).
- Добавить в Helm-чарт своего приложения шаблон ServiceMonitor (CRD monitoring.coreos.com/v1), чтобы Prometheus-оператор автоматически подхватывал метрики без ручного редактирования конфига.
- Добавить в чарт шаблон PrometheusRule с минимум двумя алертами (например: высокий процент ошибок 5xx, приложение недоступно).
- Настроить дашборд Grafana через раздел dashboards в monitoring-values.yaml (provisioning при установке, без ручного создания в UI).
- Убедиться, что helm lint ./helm/your-app проходит без ошибок.

Структура репозитория

Файл / папка	Назначение
app/	Исходный код приложения с добавленным /metrics
helm/your-app/Chart.yaml	Метаданные чарта приложения
helm/your-app/values.yaml	Значения по умолчанию для чарта
helm/your-app/templates/...	Существующие шаблоны (Deployment, Service и др.)
helm/your-app/templates/servicemonitor.yaml	НОВОЕ: CRD ServiceMonitor
helm/your-app/templates/prometheusrule.yaml	НОВОЕ: CRD PrometheusRule с алертами
helm/monitoring-values.yaml	НОВОЕ: values для kube-prometheus-stack
README.md	Инструкция по запуску + скриншоты

Ключевые требования к манифестам

Объект	Обязательные условия
Service приложения	Порт должен иметь имя (name: http или аналог) — на него ссылается ServiceMonitor
ServiceMonitor	Label release: monitoring; selector совпадает с

	labels вашего Service; указан правильный port и path: /metrics
PrometheusRule	Label release: monitoring; минимум 2 алерта с полями expr, for, labels, annotations
monitoring-values.yaml	serviceMonitorSelectorNilUsesHelmValues: false — иначе оператор не увидит ServiceMonitor из чужого namespace

Критерии оценки

Критерий	Баллы
Приложение отдаёт метрики на /metrics	1
kube-prometheus-stack задеплоен через Helm с кастомным monitoring-values.yaml	2
ServiceMonitor создан, target приложения в Prometheus в состоянии UP	2
Grafana дашборд с ≥ 3 панелями, provisioned через values (не вручную)	2
PrometheusRule создан, алерты видны в Prometheus UI → Alerts	2
helm lint ./helm/your-app проходит без ошибок	1
Итого	10

Скриншоты в README: (1) `helm list -A` — оба релиза; (2) Prometheus → Targets — ваш job UP; (3) Prometheus → Alerts — ваши алерты; (4) Grafana — дашборд с данными.